APP Mapping and Navigation

APP Mapping and Navigation

1. Course Content
2. Preparation
3. APP Mapping Case
4. APP Navigation Case
5. Source Code Analysis
    - 5.1 Launching
    - 5.2 transmission.py
    - 5.3 robot_pose_publisher.py
    - 5.4 app_save_map.py

1. Course Content

- Use the mobile APP "ROS Robot" to control the car for mapping and navigation.

2. Preparation

1. First download the `"ROS Robot"` APP on your phone. Android/iOS users can scan the QR code to download the remote control software. iOS users can also search for and download the mapping/navigation APP `"ROSRobot"` in the App Store.

2. The car, the virtual machine, and the phone must be on the same local network, which can be achieved by connecting them to the same WiFi.

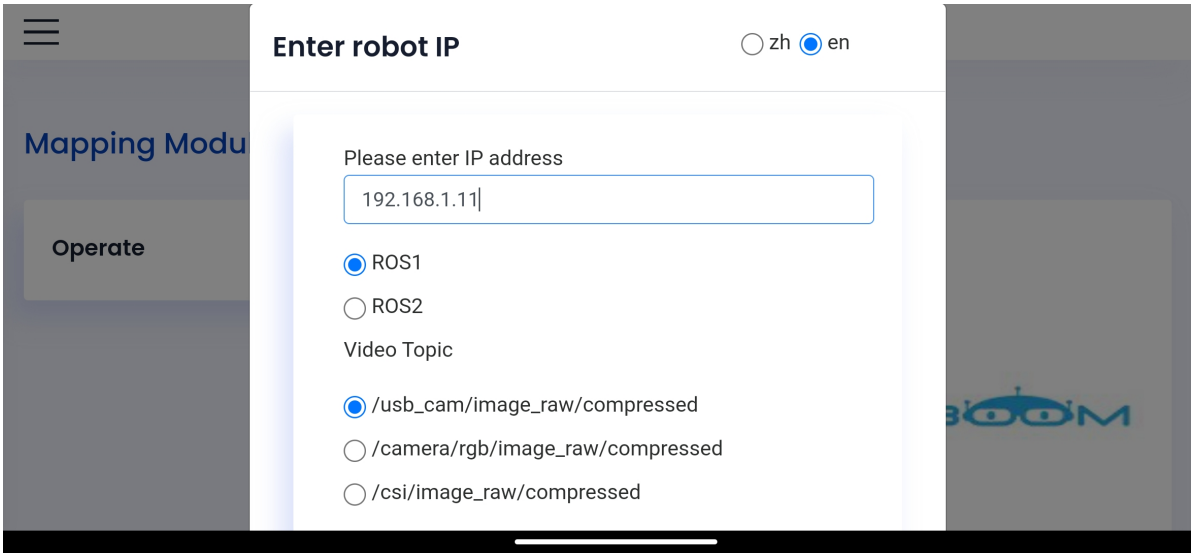
3. APP Mapping Case

- Start the service program on the virtual machine side

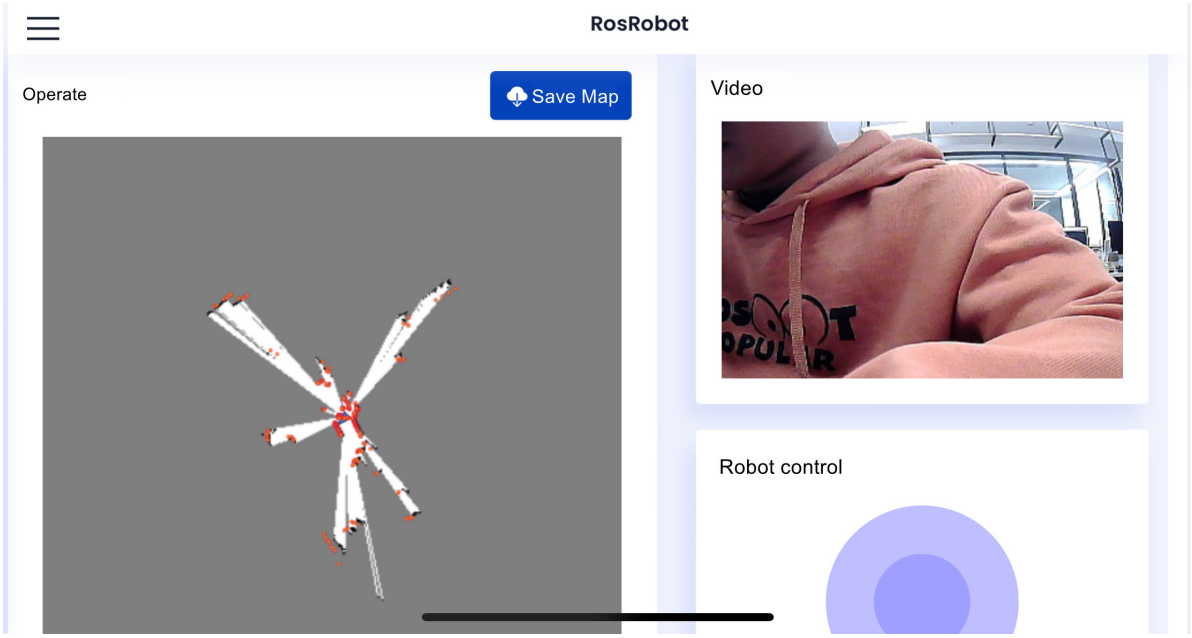
```
ros2 launch microros_v2_bringup bringup_app.launch.py slam_mode:=slam_toolbox
```

- Open `"ROS Robot"` on the phone; the display is as shown below

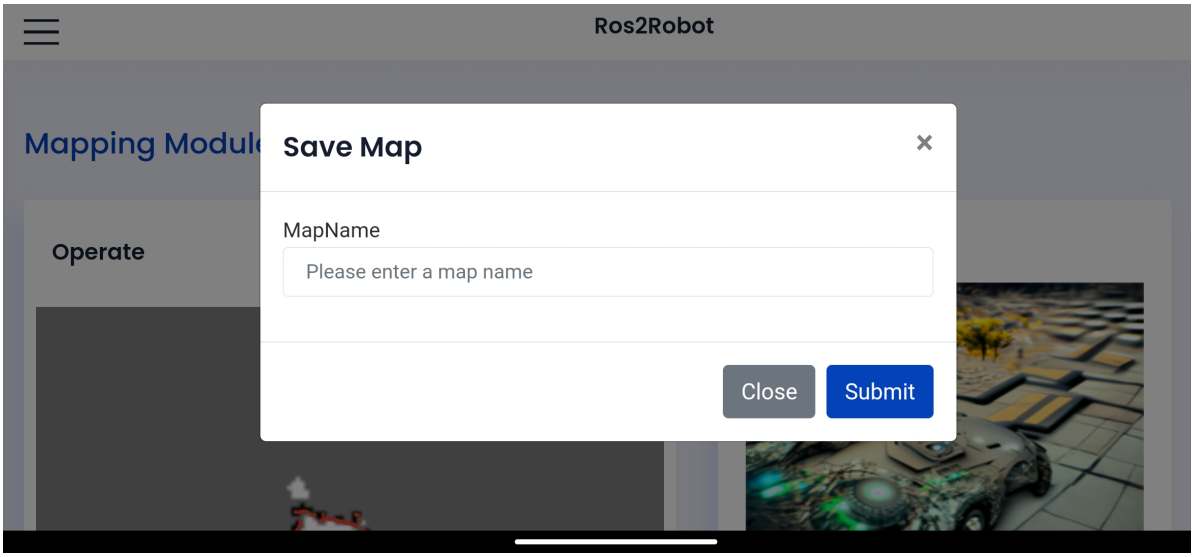
- Enter the car's IP address ——> `ROS2` ——> `VideoTpoic/camera/rgb/image_raw/compressed` ——> "Connect"



- After connecting successfully, the display is as follows



- Control the car to move slowly through the entire area to be mapped by sliding the wheel, then click Save Map, enter the map name, and click Submit to save the map.



- The map will be automatically saved to the system's `~/map` directory, and the terminal will prompt that the map was saved successfully.

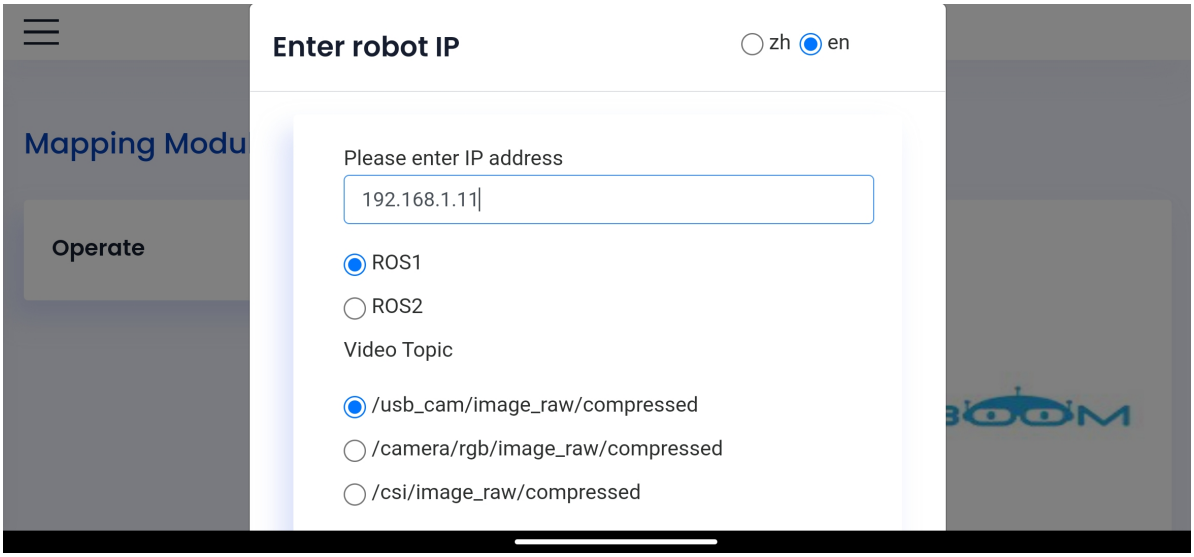
```
[server-6] [INFO] [1765334967.910089971] [map_saver]: Creating
[server-6] [INFO] [1765334967.912193848] [map_saver]: Configuring
[server-6] [INFO] [1765334967.922756528] [map_saver]: Saving map from 'map' topic to '/home/jetson/map
/4444' file
[server-6] [WARN] [1765334967.922858419] [map_saver]: Free threshold unspecified. Setting it to defaul
t value: 0.250000
[server-6] [WARN] [1765334967.922887156] [map_saver]: Occupied threshold unspecified. Setting it to de
fault value: 0.650000
[server-6] [WARN] [1765334967.951984200] [map_io]: Image format unspecified. Setting it to: pgm
[server-6] [INFO] [1765334967.954177999] [map_io]: Received a 449 X 224 map @ 0.05 m/pix
[server-6] [INFO] [1765334968.016817320] [map_io]: Writing map occupancy data to /home/jetson/map/4444
.pgm
[server-6] [INFO] [1765334968.019211798] [map_io]: Writing map metadata to /home/jetson/map/4444.yaml
[server-6] [INFO] [1765334968.019438653] [map_io]: Map saved
[server-6] [INFO] [1765334968.019460606] [map_saver]: Map saved successfully
[server-6] [INFO] [1765334968.023690856] [map_saver]: Destroying
```

4. APP Navigation Case

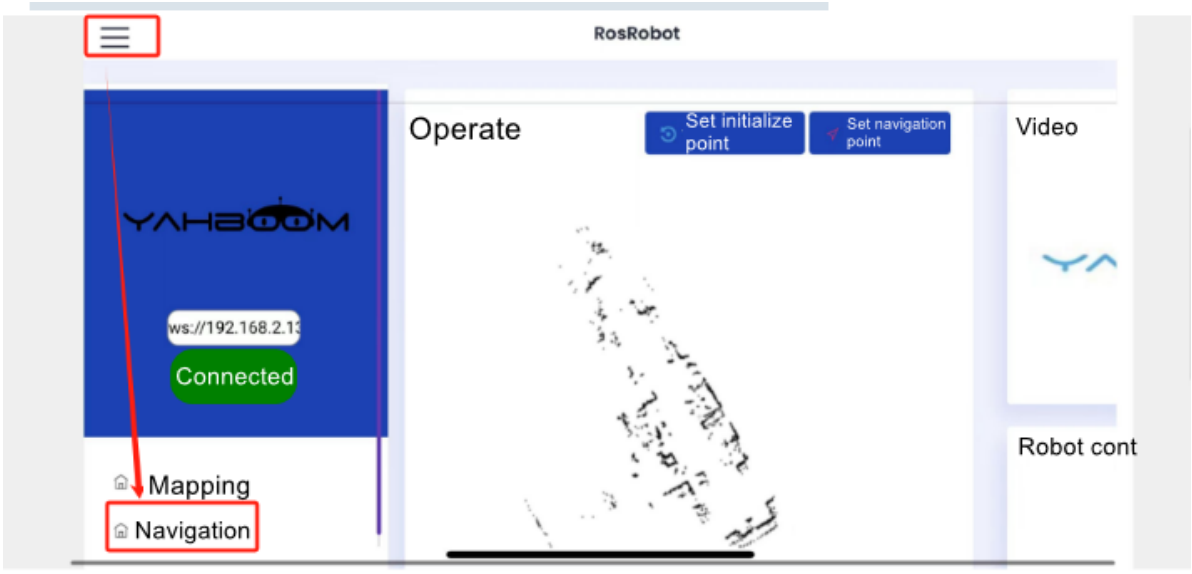
```
ros2 launch microros_v2_bringup bringup_app.launch.py navigation_mode:=True
```

> [!NOTE]
>
> **Optional parameter descriptions:**
>
> - map_file
>   - Map yaml name, default value is `map.yaml`

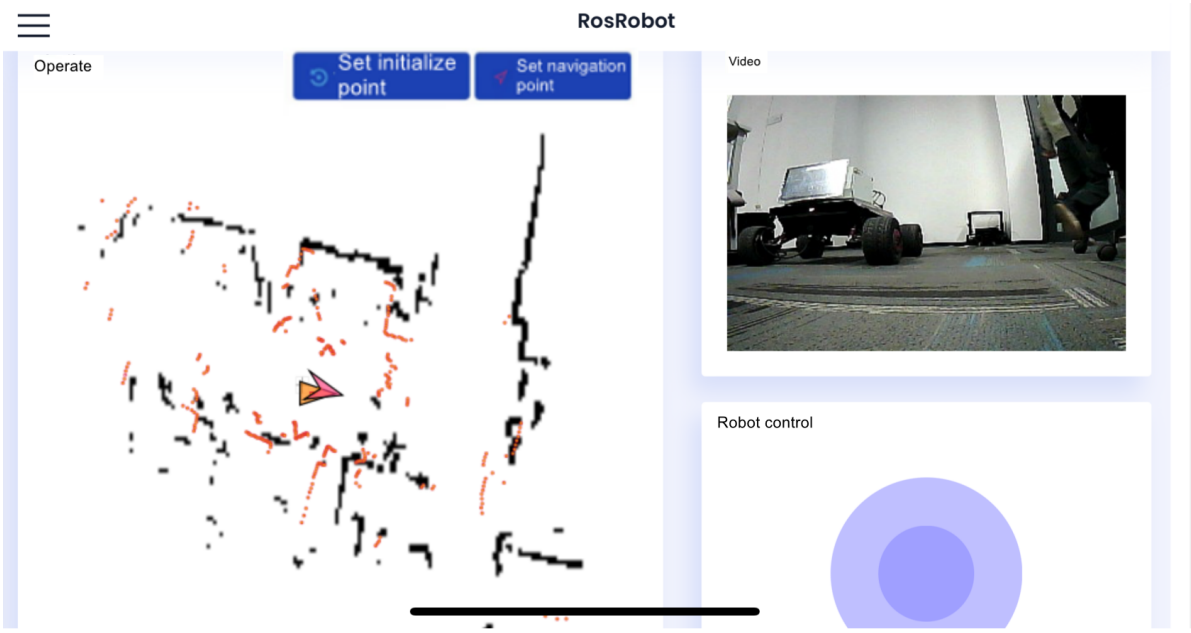
- Open `"ROS Robot"`; the display is as shown below
- Enter the car's IP address ——> `ROS2` ——> `VideoTpoic/camera/rgb/image_raw/compressed` ——> "Connect"



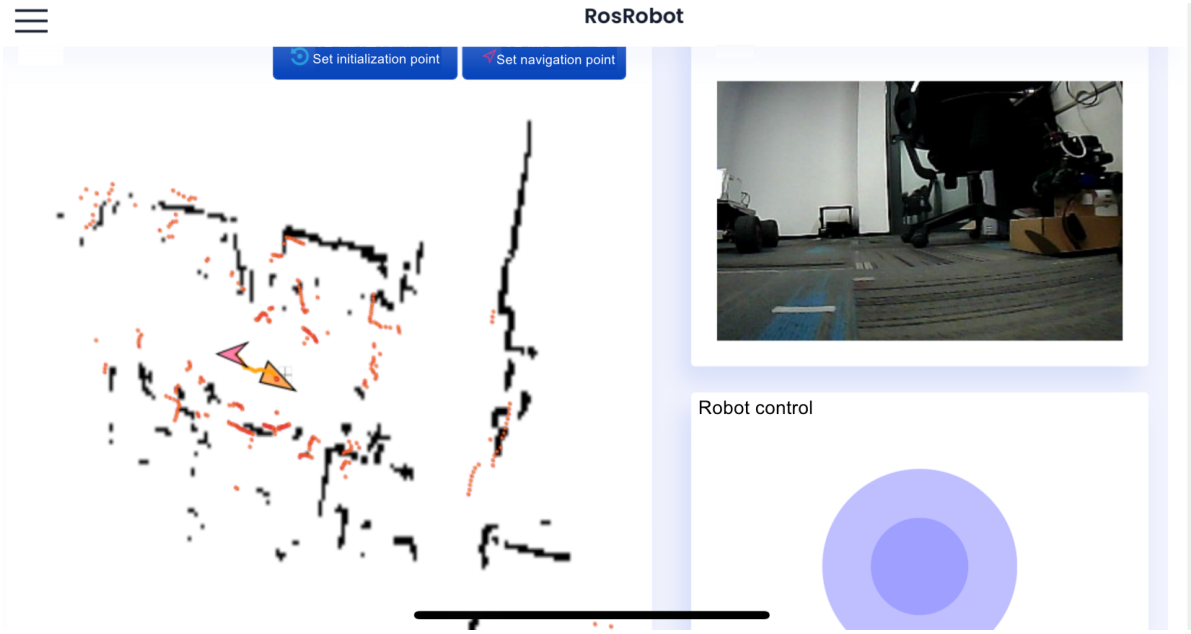
- As shown in the figure below, select the navigation interface:



- Then, based on the car's actual pose, click [Set Initialization Point] to give the car an initial pose. If the area scanned by the lidar roughly coincides with the actual obstacles, the pose is accurate. As shown in the figure below:



- Click [Set Navigation Point] to give the car a destination. The car will plan a path and move to the destination along the path. As shown in the figure below:



5. Source Code Analysis

5.1 Launching

Source code path:

```
$HOME/microros_ws/src/microros_v2_bringup/launch/bringup_app.launch.py
```

- The key programs launched are as follows:
- The program decides between mapping mode and navigation mode through the two launch parameters `slam_mode` and `navigation_mode`, and at the same time starts the rosbridge

WebSocket server, which transmits lidar, image, pose, and other data to the phone APP through the WebSocket protocol.

1. **Camera driver**: The `camera_driver` node builds the video stream URL from `camera_ip` and outputs a compressed image for display in the APP.
2. **Mapping mode branches**: Depending on the value of the `slam_mode` parameter (`gmapping` / `slam_toolbox` / `cartographer`), the corresponding mapping launch file is conditionally started, all with RViz disabled to save resources.
3. **rosbridge server**: Starts `rosbridge_websocket`, and the APP communicates with ROS 2 topics/services through WebSocket.
4. **transmission node**: Converts `LaserScan` into a `Path` point cloud and compresses the RGB image, adapting them to the APP's transmission format.
5. **robot_pose_publisher node**: Queries the `map → base_link` TF transform at 20Hz and publishes `PoseStamped` to `/robot_pose`, for the APP to display the robot's real-time pose.
6. **app_save_map node**: Provides the `yahboomAppSaveMap` service. When triggered by the APP, it calls `map_saver_cli` to save the map to the `~/map` directory and writes a record to the SQLite database.

```
    bringup_camera=Node(
        package='microros_v2_bringup',
        executable='camera_driver',
        output='screen',
        parameters=[{
            'camera_url': GetCameraUrl(),

 'img_save_path':os.path.join(os.path.expanduser('~'),'microros_ws','cache','ima
ge.png'),

 'img_save_path':os.path.join(os.path.expanduser('~'),'microros_ws','cache','ima
ge.png'),
        'ost_file':
os.path.join(get_package_share_directory('microros_v2_bringup'),'config','ost.ya
ml')}]
        )

    #gmapping
    gmapping_launch=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(bringup_dir,
                        'launch',
                        'gmapping.launch.py')),
        launch_arguments={
                        "rviz": "False",
                         }.items(),
        condition=IfCondition(PythonExpression(["'", slam_mode, "' == 'gmapping'
and '", navigation_mode, "' == 'False' "]))
    )
    #slam_toolbox
    slam_toolbox_launch=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(bringup_dir,
                        'launch',
                        'slam_toolbox.launch.py')),
        launch_arguments={
                        "rviz": "False",
                         }.items(),
```

```
        condition=IfCondition(PythonExpression(["'", slam_mode, "' ==
'slam_toolbox' and '", navigation_mode, "' == 'False' "]))
    )

    #Start Cartographer mapping mode.
    cartographer_launch=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(bringup_dir,
                        'launch',
                        'cartographer.launch.py')),
        launch_arguments={
                        "rviz": "False",
        }.items(),
        condition=IfCondition(PythonExpression(["'", slam_mode, "' ==
'cartographer' and '", navigation_mode, "' == 'False' "]))

        )

    #Start the rosbridge node.
    bringup_rosbridge_server = ExecuteProcess(
        cmd=['ros2', 'launch', 'rosbridge_server',
'rosbridge_websocket_launch.xml'],
        output='screen'
    )

    transmission_node = Node(
            package='ros_robot_app',
            executable='transmission',
            name='transmission',
            parameters=[
                {'flip_laser_points': False}
            ],
            output='screen',
    )

    #Robot pose publishing node
    robot_pose_publisher=Node(
        package="ros_robot_app",
        executable="robot_pose_publisher",
        name="robot_pose_publisher",
        output="screen",
        parameters=[
                {"use_sim_time": False},
                {"is_stamped": True},
                {"map_frame": "map"},
                {"base_frame": "base_link"}
        ]
    )

    #App map saving service node
    app_save_map_node = Node(
        package='ros_robot_app',
        executable="app_save_map",
        name="app_save_map",
        output="screen",
    )

    app_nav= IncludeLaunchDescription(
                PythonLaunchDescriptionSource(os.path.join(bringup_dir,
```

```
                                    "launch",
                                    'nav2.launch.py')),
                launch_arguments={
                    "map": LaunchConfiguration("map_file"),
                    "rviz": "False",
                    "localization_mode": "amcl",
                    "slam": "False",
                }.items(),
                condition=IfCondition(PythonExpression(["'", navigation_mode, "'
== 'True'" ]))
             )
```

5.2 transmission.py

- Source code path

```
$HOME/microros_ws/src/ros_robot_app/ros_robot_app/transmission.py
```

The `transmission` node is responsible for converting ROS 2 raw sensor data into a format the APP can receive. It mainly has two functions:

1. **Lidar data conversion (`laserscan_callback`)**: Subscribes to the `LaserScan` messages on the `/scan` topic, iterates over all distance values, filters out invalid data (inf, NaN, ≤0), converts polar coordinates to Cartesian coordinates `(x, y)`, packs them into a `Path` message and publishes it. If `flip_laser_points` is `True`, a 180° offset is added to compensate for the lidar mounting direction.
2. **Image compression (`rgb_image_callback`)**: Subscribes to the raw RGB image, converts it to JPEG compressed format via OpenCV (quality is controlled by the `jpeg_quality` parameter), and publishes a `CompressedImage` to the APP to reduce WebSocket transmission bandwidth.

```
    def laserscan_callback(self, msg):
        """Process LaserScan data"""
        angle_min = msg.angle_min
        angle_increment = msg.angle_increment
        laserscan = msg.ranges

        laser_points = self.laserscan_to_points(laserscan, angle_min,
angle_increment, msg.header)
        self.scan_point_publisher.publish(laser_points)

    def laserscan_to_points(self, laserscan, angle_min, angle_increment,
header):
        """Convert LaserScan data to point cloud path"""
        points = []
        angle = angle_min
        laser_points = Path()

        laser_points.header = header
        laser_points.header.frame_id = 'laser_frame'

        # Add 180° offset if flip is enabled to compensate for laser_link to
laser TF rotation
```

```
        angle_offset = math.pi if self.flip_laser_points else 0.0

        for distance in laserscan:
            # Filter invalid distances (infinity, NaN, out of range)
            if math.isinf(distance) or math.isnan(distance) or distance <= 0:
                angle += angle_increment
                continue

            # Convert polar coordinates to Cartesian coordinates with optional
180° offset
            adjusted_angle = angle + angle_offset
            x = distance * math.cos(adjusted_angle)
            y = distance * math.sin(adjusted_angle)

            pose = PoseStamped()
            pose.header = laser_points.header
            pose.pose.position.x = x
            pose.pose.position.y = y
            points.append(pose)

            angle += angle_increment

        laser_points.poses = points
        return laser_points

    # ========== RGB Image Compression ==========

    def rgb_image_callback(self, msg):
        """Process RGB image and publish compressed version"""
        try:
            # Convert ROS Image message to OpenCV format
            cv_image = self.rgb_bridge.imgmsg_to_cv2(msg,
desired_encoding='bgr8')

            # Set JPEG compression parameters
            encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), self.jpeg_quality]

            # Compress image
            result, encoded_img = cv2.imencode('.jpg', cv_image, encode_param)

            if not result:
                self.get_logger().error("Failed to compress image")
                return

            # Create compressed image message
            compressed_msg = CompressedImage()
            compressed_msg.header = msg.header
            compressed_msg.format = 'jpeg'
            compressed_msg.data = encoded_img.tobytes()

            # Publish compressed image
            self.rgb_img_pub.publish(compressed_msg)

        except Exception as e:
            self.get_logger().error(f"Error in RGB image callback: {e}")
```

5.3 robot_pose_publisher.py

- Source code path

```
$HOME/microros_ws/src/ros_robot_app/ros_robot_app/robot_pose_publisher.py
```

The `robot_pose_publisher` node continuously queries the TF transform at 20Hz and publishes the robot pose to the `/robot_pose` topic for the APP:

1. **Parameter configuration**: Supports three parameters: `map_frame` (default `map`), `base_frame` (default `base_link`), and `is_stamped` (whether to use `PoseStamped`).
2. **TF query**: The `timer_callback` queries the `map → base_link` transform through the `TransformListener` every 50ms, extracting the translation and rotation quaternion into a pose message and publishing it. If the TF is not yet available (`LookupException`), it is silently skipped without reporting an error.
3. **APP usage**: After receiving `/robot_pose`, the APP marks the robot's current position and orientation on the map, which is used for localization initialization and navigation status display.

```
class RobotPosePublisher(Node):
    def __init__(self):
        super().__init__('robot_pose_publisher')

        # Create TF buffer and listener
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)

        # Declare parameters
        self.declare_parameter('map_frame', 'map')
        self.declare_parameter('base_frame', 'base_link')
        self.declare_parameter('is_stamped', False)

        # Get parameters
        self.map_frame =
self.get_parameter('map_frame').get_parameter_value().string_value
        self.base_frame =
self.get_parameter('base_frame').get_parameter_value().string_value
        self.is_stamped =
self.get_parameter('is_stamped').get_parameter_value().bool_value

        # Create publisher based on is_stamped parameter
        if self.is_stamped:
            self.publisher = self.create_publisher(PoseStamped, 'robot_pose', 1)
        else:
            self.publisher = self.create_publisher(Pose, 'robot_pose', 1)

        # Create timer (50ms = 20Hz)
        self.timer = self.create_timer(0.05, self.timer_callback)

        self.get_logger().info(f'Robot Pose Publisher started')
        self.get_logger().info(f'  map_frame: {self.map_frame}')
        self.get_logger().info(f'  base_frame: {self.base_frame}')
        self.get_logger().info(f'  is_stamped: {self.is_stamped}')

    def timer_callback(self):
```

```
        try:
            # Lookup transform from map to base_link
            # Use latest available transform (time=None or rclpy.time.Time())
            transform_stamped = self.tf_buffer.lookup_transform(
                self.map_frame,
                self.base_frame,
                rclpy.time.Time()
            )

            # Create pose from transform
            if self.is_stamped:
                pose_msg = PoseStamped()
                pose_msg.header.frame_id = self.map_frame
                pose_msg.header.stamp = self.get_clock().now().to_msg()

                pose_msg.pose.position.x =
transform_stamped.transform.translation.x
                pose_msg.pose.position.y =
transform_stamped.transform.translation.y
                pose_msg.pose.position.z =
transform_stamped.transform.translation.z

                pose_msg.pose.orientation.x =
transform_stamped.transform.rotation.x
                pose_msg.pose.orientation.y =
transform_stamped.transform.rotation.y
                pose_msg.pose.orientation.z =
transform_stamped.transform.rotation.z
                pose_msg.pose.orientation.w =
transform_stamped.transform.rotation.w

                self.publisher.publish(pose_msg)
            else:
                pose_msg = Pose()
                pose_msg.position.x = transform_stamped.transform.translation.x
                pose_msg.position.y = transform_stamped.transform.translation.y
                pose_msg.position.z = transform_stamped.transform.translation.z

                pose_msg.orientation.x = transform_stamped.transform.rotation.x
                pose_msg.orientation.y = transform_stamped.transform.rotation.y
                pose_msg.orientation.z = transform_stamped.transform.rotation.z
                pose_msg.orientation.w = transform_stamped.transform.rotation.w

                self.publisher.publish(pose_msg)

        except LookupException as e:
            # TF not available yet, silently skip
            pass
        except Exception as e:
            self.get_logger().warn(f'Error looking up transform: {e}')
```

5.4 app_save_map.py

- Source code path

```
$HOME/microros_ws/src/ros_robot_app/ros_robot_app/app_save_map.py
```

The `app_save_map` node provides a ROS 2 service `yahboomAppSaveMap` that receives the APP's map saving request:

1. **Service callback (`add_three_ints_callback`)**: Receives the `mapname` parameter, concatenates the full save path `~/map/<map_name>`, and generates a timestamped MD5 hash as the map ID.
2. **Database record**: Connects to the SQLite database `xgo.db`, writes the map name, ID, and path into the `xgo_map` table; if the map name is duplicated and an `IntegrityError` is triggered, an error message is returned.
3. **Map saving**: Calls `ros2 run nav2_map_server map_saver_cli` via `subprocess` to save the map files (`.yaml` + `.pgm`), with a timeout set to 10000 seconds to ensure large maps can be saved completely.

```
class MinimalService(Node):

    def __init__(self):

        super().__init__('minimal_service')
        self.srv = self.create_service(WebSaveMap, 'yahboomAppSaveMap',
self.add_three_ints_callback)       # CHANGE

    def run_shellcommand(self, *args):
        '''run the provided command and return its stdout'''
        args = sum([(arg if type(arg) == list else [arg]) for arg in args], [])
        print(args)
        possess = subprocess.Popen(args, stdout=subprocess.PIPE)
        return possess

    def add_three_ints_callback(self, request, response):
        map_name = request.mapname
        map_path =os.path.join(map_folder_dir, map_name)
        now = datetime.datetime.now()
        str_time = now.strftime("%Y-%m-%d %H:%M:%S.%f")
        map_namestr = str_time + map_name
        map_id = hashlib.md5(map_namestr.encode()).hexdigest()
        response.response = request.mapname
        try:
            conn = sqlite3.connect(os.path.join(map_folder_dir, '.data',
'xgo.db'))
            c = conn.cursor()
            c.execute("INSERT INTO xgo_map (map_name, map_id, map_path) VALUES
(?, ?, ?)",(map_name, map_id, map_path))
            self.run_shellcommand('ros2', 'run', 'nav2_map_server',
'map_saver_cli','-f', map_path, '--ros-args', '-p',
'save_map_timeout:=10000.00')
        except sqlite3.IntegrityError as e:
            re_data = {"msg": str(e)}
            response.response = re_data
```

```
            self.get_logger().info('Incoming request\nmapname: %s' % (re_data))
 # CHANGE
            return response
                                                                # CHANGE
        self.get_logger().info('Incoming request\nmapname: %s' %
(request.mapname))  # CHANGE

        return response
```